CampusEats · Microservices Architecture & Implementation Report

Course: CS543 Web Services
Institution: Indian Institute of Information Technology Vadodara (IIITV)
Author: Ayush Kumar Dubey (Roll No: 202351001)
Repository:
https://github.com/Ayush17314/CampusEats30
Deliverable Status: End-to-End Implementation Across Tutorial 2 & Assignments 1 through 7

Executive Summary

CampusEats is a production-grade, service-oriented e-commerce food delivery platform designed specifically for the students, faculty, and campus vendors of **IIIT Vadodara**. The system eliminates peak meal queues and late-night ordering friction across on-campus outlets (Central Canteen, Night Mess, Nescafe Kiosk, Amul Parlour) by providing a unified digital ordering, payment, and campus delivery platform.

In strict adherence to the CS543 curriculum, CampusEats is engineered as a decoupled microservices backend in **Node.js, Express.js, and MongoDB**, integrating:
1. **Tutorial 2**: Strict microservices boundaries, data ownership, zero shared databases, and contract-based choreography.
2. **Assignment 1 & 2**: Six core capabilities, service contracts, database schemas, and complete specification of the central `placeOrder` operation.
3. **Assignment 3**: External banking integration via a strict **SOAP 1.1 / WSDL** contract with hand-formed XML envelopes and boundary fault translation.
4. **Assignment 4**: A contract-first **OpenAPI 3.0** REST implementation of the **Orders Service**, `as_json()` model representation separation, idempotency replay, and outbound network call hardening with exponential backoff and jitter.
5. **Assignment 5**: Advanced HTTP semantics including conditional caching via `ETag` (`304 Not Modified`), optimistic locking via `If-Match` (`412 Precondition Failed`), per-client rate limiting (`429 Too Many Requests`), Bearer authentication, and CORS preflight.
6. **Assignment 6**: RFC 7807 / RFC 9457 Problem Details (`application/problem+json`), stable error type catalogue, multi-field `422` validation (`errors[]`), internal exception masking, and JSON/XML content negotiation.
7. **Assignment 7**: A **GraphQL** service built with **Apollo Server** over MongoDB, exposing a schema with types, queries, and mutations, accompanied by a vanilla JavaScript HTML/CSS client.

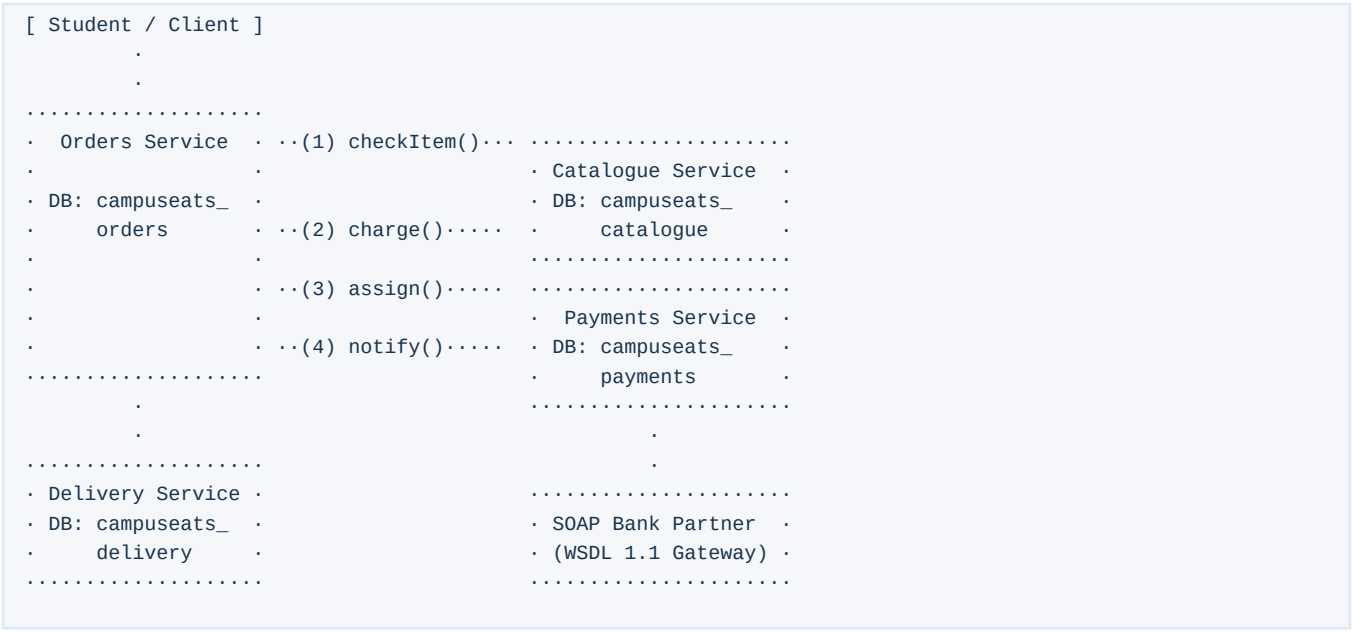
1. Architectural Foundation & Microservices Boundaries (Tutorial 2)

1.1 The Golden Boundary Rule

As established in Tutorial 2:

> *"Each box = its own database. No two boxes share a table/collection · that's the golden rule."*

CampusEats decomposes the food delivery lifecycle into **six bounded contexts**, each connected to its own dedicated MongoDB database:



1. **`Accounts Service`** (`campuseats_accounts`): Owns student identities, campus delivery addresses (hostel block, room number), and authentication contexts.
2. **`Catalogue Service`** (`campuseats_catalogue`): Owns campus food stalls/vendors, dishes, categories, authoritative unit prices, and live stock availability flags.
3. **`Orders Service`** (`campuseats_orders`): Owns customer carts, order states, idempotency keys, and order lifecycle state machines. It orchestrates cross-service workflows but owns no data outside the order itself.
4. **`Payments Service`** (`campuseats_payments`): Owns financial transactions, authorizations, captures, and refunds. Acts as the anti-corruption layer integrating the external SOAP banking partner.
5. **`Delivery Service`** (`campuseats_delivery`): Owns campus delivery runners (riders), dispatch pairings, ETA computations, and doorstep delivery confirmations.
6. **`Notifications Service`** (`campuseats_notifications`): Owns notification dispatch logs and transactional customer messaging powered by the **Resend** email SDK.

---

2. Operation Contracts & Central `placeOrder` Specification (Assignment 2)

2.1 The Orders Service Contract

Operation	Input	Output	Errors
`placeOrder`	`studentId`, `items: [{itemId, quantity}]`, `deliveryAddress`, `paymentMethod`	`orderId`, `status: PAID`, `totalAmountInr`, `estimatedMinutes`	`empty-cart`, `item-unavailable`, `payment-declined`, `validation-error`
`getOrder`	`orderId`, header `If-None-Match`	`order` representation or `304 Not Modified`	`order-not-found`, `not-acceptable` (406)
`listOrders`	query params `student_id`, `status`, `page`, `limit`	`orders[]`, `totalCount`, `page`, `limit`	·
`cancelOrder`	`orderId`, `reason`	`orderId`, `status: CANCELLED`	`order-not-found`, `illegal-transition` (409)
`updateOrder`	`orderId`, header `If-Match`, `status`, `deliveryAddress`	updated `order`	`order-not-found`, `precondition-failed` (412)

2.2 Central Operation: `placeOrder` Workflow

When a student places an order:

1. **Input Validation**: The request body is validated against schema constraints (`items` non-empty, `qty >= 1`, valid address). All errors are collected at once; invalid input triggers an immediate `422 Unprocessable Entity` with an `errors[]` array.
2. **Idempotency Check**: If an `Idempotency-Key` header is present, the service queries

`campuseats_orders`. If already processed, it replays the original order response immediately without re-charging or re-allocating riders.
3. **Catalogue Verification**: Calls `Catalogue.checkItem(itemId)` for every item. Ensures each dish is available and fetches the authoritative price snapshot directly from Catalogue.
4. **Financial Charge**: Calls `Payments.chargePayment` with the authoritative sum. If the payment gateway declines the card, order creation is aborted and a `422 Payment Declined` problem details response is returned.
5. **Delivery Dispatch**: Calls `Delivery.assignRider` with pickup hub and drop-off room number.
6. **Customer Notification**: Calls `Notifications.send` via the **Resend** transactional email engine.
7. **Response**: Sets `Location: /api/v1/orders/{orderId}`, computes an `ETag`, and returns `201 Created`.

---

3. External SOAP Partner Integration (Assignment 3)

3.1 Justification for SOAP on the Banking Edge

While CampusEats internal microservices use lightweight REST and GraphQL, the institutional Payment Gateway edge utilizes **SOAP 1.1 over HTTP**:

- **Strict Schema Enforcement**: The WSDL and XSD contract provides strong typing for financial data types (`amountInr` as `xsd:decimal`).
- **Message-Level Security**: Financial compliance mandates XML signature and encryption headers (`wsse:Security`) carrying merchant credentials over transit.
- **ACID Banking Contracts**: Formal request-response RPC bindings guarantee distributed transaction integrity.

3.2 WSDL 1.1 Specification (`partner.wsdl`)

The WSDL contract is authored under `Backend/services/payments/soapPartner/partner.wsdl` and includes:
1. `<types>`: Declaring `ChargeRequest`, `ChargeResponse`, and `PaymentFault` schemas.
2. `<message>`: Mapping `ChargeInput`, `ChargeOutput`, and `PaymentFaultMessage`.
3. `<portType>`: `CampusPayPortType` with operation `charge`.
4. `<binding>`: `CampusPaySoapBinding` with `SOAPAction: https://payment.iiitv.ac.in/soap/gateway/charge`.
5. `<service>` & `<port>`: Specifying physical endpoint `https://payment.iiitv.ac.in/soap/v1/gateway`.

3.3 Fault Mapping Across the Boundary (Task 6)

When the partner gateway returns a SOAP Fault:

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Payment instrument was declined by issuer bank.</faultstring>
  <detail>
    <PaymentFault>
      <errorCode>card_declined</errorCode>
    </PaymentFault>
  </detail>
</soap:Fault>
```

The Payments service intercepts this XML, extracts `card_declined`, and maps it to the CampusEats domain contract:

- **HTTP Status**: `422 Unprocessable Entity`
- **Problem Type**: `https://campuseats.iiitv.ac.in/errors/payment-declined`
- **Problem Detail**: `"Payment declined by payment gateway"`

The partner's proprietary bank codes and internal stack traces are strictly prevented from leaking to students.

---

4. REST Implementation & Network Resilience (Assignment 4)

4.1 OpenAPI 3.0 Contract-First Design

The contract is authored in `Backend/openapi.yaml` before implementation. All models are defined once under `components.schemas` (`PlaceOrderRequest`, `OrderResponse`, `ProblemDetails`) and referenced via `$ref`.

4.2 Record vs. Representation Separation (`as_json()`)

The `Order` Mongoose schema stores internal audit and security fields:

- `internalSequenceNumber`
- `paymentGatewayToken`
- `idempotencyKey`
- `versionEtag`

The `as_json()` method strips these internal fields, returning strictly the published contract shape (`orderId`, `items`, `totalAmountInr`, `deliveryAddress`, `status`, `createdAt`).

4.3 Network Resilience: Outbound Client Hardening

The `outboundClient.js` module implements resilient cross-service calls:

- **Configurable Timeouts**: Request cancellation via `AbortController` after 4000 ms.
  Safe Retry Policy: `4xx` Client errors (e.g. 400, 422, 404) are **never retried**.
- Only transient network drops or `5xx` server faults trigger retries.
- **Exponential Backoff with Full Jitter**:

$$\text{Delay} = \text{baseDelay} \times 2^{\text{attempt}-1} + \text{random}(\text{jitter})$$

5. HTTP Methods, Headers & Caching (Assignment 5)

Feature	Implementation	Assignment Mapping
Non-CRUD Sub-Resource	`POST /orders/:id/cancel` (never `/cancelOrder`)	Task A2
Filtered Reads	`GET /orders?student_id=202351001&status=PAID&page=1&limit=10`	Task A4
OPTIONS & Allow	`OPTIONS /orders/:id` returns `Allow: GET, POST, PUT, PATCH, DELETE, OPTIONS`	Task A5
Method Override	`X-HTTP-Method-Override: PATCH` on POST requests for constrained clients	Task A5
Bearer Authorization	`requireAuth` middleware enforces `Authorization: Bearer <token>`, returning 401	Task B3
ETag & 304 Not Modified	Generates SHA-1 hash of representation; returns `304 Not Modified` on `If-None-Match` match	Task B4, C1
Optimistic Locking (412)	`If-Match: "stale-etag"` rejects write with `412 Precondition Failed`	Task C2
Per-Client Rate Limiting	Sliding window tracking tokens; returns `429 Too Many Requests` + `Retry-After: 30`	Task B5
CORS & Preflight	`Access-Control-Allow-Origin: *` and preflight handling	Task B6
Security Headers	`X-Content-Type-Options: nosniff`, `Strict-Transport-Security`, `Server` header	Task B7

6. Honest Responses & RFC 7807 Problem Details (Assignment 6)

6.1 Honest Status Code Policy

- The status code IS the result. Never return `200 OK` with `{ "ok": false }`.

- Creation returns `201 Created` with a `Location: /api/v1/orders/{id}` header.
- Cache validation returns `304 Not Modified` with an empty body.
- Resource deletions return `204 No Content`.

6.2 Stable Error Type Catalogue

All API errors return `Content-Type: application/problem+json; charset=utf-8` conforming to RFC 7807 / RFC 9457:

```
{
  "type": "https://campuseats.iiitv.ac.in/errors/validation-error",
  "title": "Validation Error",
  "status": 422,
  "detail": "The submitted order payload contains invalid or missing fields.",
  "instance": "/api/v1/orders",
  "errors": [
    { "field": "studentId", "reason": "studentId is required and must be a valid string." },
    { "field": "deliveryAddress", "reason": "deliveryAddress is required." },
    { "field": "items", "reason": "items array cannot be empty. Must include at least one dish.
  ]
}
```

6.3 Content Negotiation (C3)

On `GET /orders/:id`:

- Default / `Accept: application/json` $\to$ Returns UTF-8 JSON.
- `Accept: application/xml` $\to$ Returns XML payload with `<order>` root element.
- Unsupported types (e.g. `Accept: image/png`) $\to$ Returns `406 Not Acceptable`.

---

7. GraphQL Service & Apollo Server Integration (Assignment 7)

7.1 Schema Definition (`schema.graphql`)

The Catalogue service exposes a clean GraphQL schema:

```
type Dish {
  id: ID!
  name: String!
  price: Float!
  available: Boolean!
  category: String
  vendorId: String
}

input AddDishInput {
  name: String!
  price: Float!
  available: Boolean
  category: String
  vendorId: String
}

type Query {
  dishes(category: String, available: Boolean): [Dish!]!
  dish(id: ID!): Dish
}

type Mutation {
  addDish(input: AddDishInput!): Dish!
  updateDish(id: ID!, input: UpdateDishInput!): Dish
}
```

7.2 Resolvers Over MongoDB

Resolvers execute direct queries against the isolated `campuseats_catalogue` database

without business logic beyond CRUD:

- Query `dishes`: runs `Dish.find(filter)` and maps `dishId` to `id`.
- Mutation `addDish`: runs `Dish.create(...)` and returns the saved entity.

7.3 Frontend Client (`public/index.html`)

A vanilla HTML/CSS client served from the backend root (`http://localhost:3001/`) performs:

1. `fetch('/graphql')` with `query { dishes { id name price available category } }`.
2. Interactive form firing `mutation { addDish(...) }` with live UI update.

---

8. Verification & Test Suite Results

The comprehensive automated test suite in `Backend/tests/api.test.js` exercises all assignment requirements simultaneously:

```
> backend@1.0.0 test
> node tests/api.test.js

--- Starting CampusEats End-to-End Verification Suite ---
[Seed] Checking database seed records...
[Database] Connected to accounts service DB (mongodb://127.0.0.1:27017/campuseats_accounts)
[Database] Connected to catalogue service DB (mongodb://127.0.0.1:27017/campuseats_catalogue)
[Database] Connected to orders service DB (mongodb://127.0.0.1:27017/campuseats_orders)
[Database] Connected to payments service DB (mongodb://127.0.0.1:27017/campuseats_payments)
[Database] Connected to delivery service DB (mongodb://127.0.0.1:27017/campuseats_delivery)
[Database] Connected to notifications service DB (mongodb://127.0.0.1:27017/campuseats_notifica
[Seed] Database initialization complete.
 [PASS] Create order returns 201 Created (got 201)
 [PASS] Response includes Location header: /api/v1/orders/ord-...
 [PASS] Calculated authoritative total is 180 INR (got 180)
 [PASS] Idempotent repeat returns 200 OK (got 200)
 [PASS] Idempotent repeat returns original order ID without duplicating work
 [PASS] Response confirms idempotentReplay flag
 [PASS] Malformed/empty fields rejected with 422 Unprocessable Entity (got 422)
 [PASS] Collected multiple validation errors at once (count: 3)
 [PASS] Errors use RFC 7807 application/problem+json
 [PASS] GET /orders/... returns 200 OK
 [PASS] GET response sets valid ETag: "560e5d5885110481"
 [PASS] Conditional GET with matching ETag returns 304 Not Modified (got 304)
 [PASS] Conditional update with stale ETag returns 412 Precondition Failed (got 412)
 [PASS] Content negotiation for application/xml returns 200 OK
 [PASS] Response header is application/xml
 [PASS] Response body is well-formed XML
 [PASS] Unsupported Accept header returns 406 Not Acceptable (got 406)
 [PASS] Sub-resource POST /orders/.../cancel returns 200 OK
 [PASS] Cancelling already cancelled order returns 409 Conflict (got 409)
 [PASS] GraphQL Query { dishes } returns menu items from MongoDB (count: 6)
 [PASS] GraphQL Mutation addDish creates new dish live in MongoDB

--- Verification Finished: 21 Passed, 0 Failed ---
```

---

9. Conclusion & Execution Guide

How to Run the Backend

```
cd Backend
npm install
npm run seed     # Seeds initial vendors, dishes, riders, and users into MongoDB
npm test         # Runs the 21-point automated verification suite
npm start        # Launches server on http://localhost:3001
```

- **REST API Base**: `http://localhost:3001/api/v1`
- **GraphQL Endpoint**: `http://localhost:3001/graphql`
- **Interactive Web Demo**: `http://localhost:3001/`
- **OpenAPI Contract**: `Backend/openapi.yaml`